

Thank you for giving me this opportunity to introduce myself. My name is Vijay C. S. I am currently pursuing my final year of Engineering in Information Science and Engineering at SJB Institute of Technology. I am also working as an intern at Cognizant, where I have been assigned to the IoT Business Unit.

Python Interview Questions & Answers

1. Datatypes, Operators, Basic Operations

1. What are the built-in data types in Python?

Python has built-in data types like:

- Numeric: int, float, complex
- Sequence: list, tuple, range, str
- Mapping: dict
- Set types: set, frozenset
- Boolean: bool
- Binary: bytes, bytearray, memoryview
- Special: NoneType

Interview answer:

Python provides several built-in data types such as integers, floats, strings, lists, tuples, dictionaries, sets, booleans, and None. These are used to store different kinds of data.

2. What is the difference between int and float?

int stores whole numbers, while float stores decimal numbers.

```
1 a = 10      # int
2 b = 10.5    # float
```

Interview answer:

int represents whole numbers without decimal points, whereas float represents numbers with decimal points. Floats may have precision limitations because they are stored using floating-point representation.

3. How does Python handle type conversion?

Python supports:

- **Implicit conversion**
- **Explicit conversion**

```
1 x = 10
2 y = 2.5
3 z = x + y    # int converted to float automatically

1 a = int("10")
2 b = float("3.14")
```

Interview answer:

Python can convert types automatically in some cases, such as converting an integer to a float during arithmetic. We can also manually convert types using functions like `int()`, `float()`, `str()`, and `list()`.

4. What is the difference between `==` and `is`?

`==` compares values.

`is` compares object identity/memory reference.

```
1 a = [1, 2]
2 b = [1, 2]
3
4 print(a == b)  # True
5 print(a is b)  # False
```

Interview answer:

`==` checks whether two objects have the same value, while `is` checks whether both variables point to the same object in memory.

5. Explain mutable vs immutable data types.

Mutable objects can be changed after creation. Immutable objects cannot be changed.

Mutable examples:

```
1 list, dict, set
```

Immutable examples:

```
1 int, float, str, tuple, bool
```

Interview answer:

Mutable objects like lists and dictionaries can be modified after creation. Immutable objects like strings, tuples, and integers cannot be changed once created.

6. How does Python handle large integers?

Python integers have arbitrary precision. They can grow as large as memory allows.

```
1 x = 999999999999999999999999999999
```

Interview answer:

Python does not have a fixed limit for integer size like some other languages. It automatically allocates more memory for large integers, limited only by available system memory.

7. What are bitwise operators? Where are they used?

Bitwise operators work at the binary level.

Common operators:

```
1  &    AND
2  |    OR
3  ^    XOR
4  ~    NOT
5  <<   left shift
6  >>   right shift
```

Interview answer:

Bitwise operators operate on bits. They are used in low-level programming, permissions, flags, optimization, encryption, and network programming.

8. How does Python store strings internally?

Python 3 stores strings as Unicode.

Interview answer:

Python strings are stored as Unicode internally, which allows Python to support characters from multiple languages and symbols.

9. Why are strings immutable in Python?

Strings are immutable for security, performance, and hashability.

Interview answer:

Strings are immutable so they can be safely reused, cached, and used as dictionary keys. Immutability also improves performance and prevents accidental modification.

2. Flow Control & Control Statements

10. Explain if, elif, and else.

They are conditional statements.

```
1  if marks >= 90:
2      print("A")
3  elif marks >= 75:
4      print("B")
5  else:
6      print("C")
```

Interview answer:

if checks the first condition, elif checks additional conditions, and else runs when none of the previous conditions are true.

11. What does the pass statement do?

pass does nothing. It is a placeholder.

```
1  if True:
2      pass
```

Interview answer:

pass is used when a statement is syntactically required but no action is needed. It is commonly used in empty functions, classes, or loops.

12. What is a conditional expression?

It is Python's ternary operator.

```
1  result = "Pass" if marks >= 40 else "Fail"
```

Interview answer:

A conditional expression allows writing simple if-else logic in one line.

13. Difference between break and continue.

break exits the loop.

continue skips the current iteration.

```
1  for i in range(5):
2      if i == 3:
3          break
```

```
1 for i in range(5):
2     if i == 3:
3         continue
```

Interview answer:

break stops the loop completely, while continue skips the current iteration and moves to the next one.

14. How does short-circuit evaluation work?

Python stops evaluating once the result is known.

```
1 if x != 0 and 10 / x > 2:
2     print("Valid")
```

Interview answer:

In expressions using and and or, Python evaluates only as much as needed. For and, if the first condition is false, the second is not checked. For or, if the first is true, the second is skipped.

15. Can else be used with loops?

Yes.

```
1 for i in range(3):
2     print(i)
3 else:
4     print("Loop completed")
```

The else block runs if the loop completes without break.

Interview answer:

Yes, Python allows else with loops. The else block executes only when the loop finishes normally without encountering a break.

16. Difference between for and while loop.

for is used for iteration over a sequence.

while runs while a condition is true.

Interview answer:

A for loop is generally used when the number of iterations is known or when iterating over collections. A while loop is used when execution depends on a condition.

17. How does range() work?

range() generates a sequence of numbers lazily.

```
1 range(start, stop, step)
1 for i in range(1, 10, 2):
2     print(i)
```

Interview answer:

range() returns an immutable sequence of numbers. It does not create all numbers in memory at once, so it is memory efficient.

18. What is a nested loop?

A loop inside another loop.

```
1 for i in range(3):
2     for j in range(2):
3         print(i, j)
```

Interview answer:

A nested loop is used when we need to perform repeated operations inside another repeated operation, such as working with matrices.

19. How do you iterate over a dictionary?

```
1 d = {"a": 1, "b": 2}
2
3 for key, value in d.items():
4     print(key, value)
```

Interview answer:

We can iterate over dictionary keys, values, or key-value pairs using keys(), values(), and items().

20. How does Python optimize loop execution?

Python optimizes loops through efficient built-in functions, iterators, generators, and comprehensions.

Interview answer:

Python loops can be optimized by using built-in functions like sum(), map(), list comprehensions, generators, and avoiding unnecessary operations inside loops.

21. What are iterator objects?

An iterator is an object that returns values one at a time using `__next__()`.

```
1 nums = iter([1, 2, 3])
2 print(next(nums))
```

Interview answer:

An iterator is an object that implements `__iter__()` and `__next__()`. It allows sequential access to elements without loading everything at once.

3. Lists, Tuples & Slicing

22. Difference between list and tuple.

List is mutable. Tuple is immutable.

```
1 my_list = [1, 2, 3]
2 my_tuple = (1, 2, 3)
```

Interview answer:

Lists can be modified after creation, while tuples cannot. Tuples are generally faster and can be used as dictionary keys if all elements are hashable.

23. What is slicing?

Slicing extracts part of a sequence.

```
1 nums = [1, 2, 3, 4, 5]
2 print(nums[1:4])
```

Interview answer:

Slicing is used to access a subset of a sequence using start, stop, and step values.

24. Explain common list methods.

Common methods:

```
1 append()    # adds one element
2 extend()    # adds multiple elements
3 pop()       # removes and returns element
4 remove()    # removes by value
5 sort()      # sorts list
6 reverse()   # reverses list
```

Interview answer:

append() adds a single item, extend() adds multiple items from another iterable, and pop() removes and returns an item by index.

25. How do you reverse a list?

```
1 lst.reverse()
```

or

```
1 reversed_list = lst[::-1]
```

Interview answer:

A list can be reversed in-place using reverse() or by creating a reversed copy using slicing [::-1].

26. Why are tuples hashable?

Tuples are hashable only if all their elements are hashable.

```
1 (1, 2, "a") # hashable
2 ([1, 2], 3) # not hashable
```

Interview answer:

Tuples are immutable, so their hash value can remain constant. However, a tuple is hashable only when all elements inside it are also hashable.

27. When would you choose a tuple over a list?

Use tuple when data should not change.

Interview answer:

I would choose a tuple when I want immutable data, better performance, or when the data needs to be used as a dictionary key.

4. Dictionaries & Sets

28. What is a dictionary?

A dictionary stores key-value pairs.

```
1 person = {"name": "Vijay", "age": 25}
```

Interview answer:

A dictionary is a collection of key-value pairs. It provides fast lookup using keys.

29. How are keys stored in a dictionary?

Keys are stored using a hash table.

Interview answer:

Python dictionaries use hashing internally. Each key's hash value helps determine where the key-value pair is stored.

30. Explain set operations.

Common set operations:

```
1  a | b    # union
2  a & b    # intersection
3  a - b    # difference
4  a ^ b    # symmetric difference
```

Interview answer:

Sets are used to store unique elements and support mathematical operations like union, intersection, difference, and symmetric difference.

31. How do you handle missing keys in dictionaries?

```
1  d.get("name", "Unknown")
```

or

```
1  from collections import defaultdict
```

Interview answer:

Missing keys can be handled using `get()`, `setdefault()`, `defaultdict`, or `try-except` for `KeyError`.

32. How does hashing work in Python dictionaries?

Python computes a hash of the key and uses it to locate the value.

Interview answer:

Dictionary keys are hashed. The hash value determines the bucket where the key-value pair is stored, making lookups very fast on average.

33. Why can't lists be dictionary keys?

Lists are mutable and unhashable.

Interview answer:

Dictionary keys must be immutable and hashable. Lists can change after creation, so their hash value would not be stable.

5. Functions & Modules

34. How do you define a function?

```
1 def greet(name):  
2     return "Hello " + name
```

Interview answer:

A function is defined using the def keyword. It can accept parameters and return a value.

35. What is a return statement?

return sends a value back from a function.

Interview answer:

The return statement exits a function and sends the result back to the caller.

36. What are default arguments?

```
1 def greet(name="Guest"):  
2     print("Hello", name)
```

Interview answer:

Default arguments provide default values to parameters when no value is passed by the caller.

37. Explain *args and **kwargs.

```
1 def func(*args, **kwargs):  
2     print(args)  
3     print(kwargs)
```

Interview answer:

*args allows a function to accept multiple positional arguments, while **kwargs allows multiple keyword arguments.

38. What are closures?

A closure is a function that remembers variables from its outer scope.

```
1 def outer(x):
2     def inner(y):
3         return x + y
4     return inner
```

Interview answer:

A closure occurs when an inner function remembers and uses variables from the outer function even after the outer function has finished execution.

39. Explain Python's LEGB scope rule.

LEGB stands for:

- Local
- Enclosing
- Global
- Built-in

Interview answer:

Python searches variables in the LEGB order: local scope, enclosing scope, global scope, and finally built-in scope.

6. Exception Handling

40. What is an exception?

An exception is a runtime error.

Interview answer:

An exception is an event that interrupts normal program execution. Python provides exception handling to manage such errors gracefully.

41. Explain try and except.

```
1 try:
2     x = 10 / 0
3 except ZeroDivisionError:
4     print("Cannot divide by zero")
```

Interview answer:

Code that may raise an error is placed inside try, and the error is handled inside except.

42. Difference between except Exception and bare except.

Bare except catches everything, including system-exiting exceptions.

Interview answer:

except Exception catches most application-level exceptions. A bare except catches all exceptions, including KeyboardInterrupt and SystemExit, so it should generally be avoided.

43. What is the use of finally?

finally always executes.

```
1  try:
2      pass
3  finally:
4      print("Cleanup")
```

Interview answer:

finally is used for cleanup operations such as closing files, releasing locks, or closing database connections.

44. How do you create custom exceptions?

```
1  class InvalidAgeError(Exception):
2      pass
```

Interview answer:

Custom exceptions are created by defining a class that inherits from the built-in Exception class.

45. Explain exception chaining.

Exception chaining links one exception to another.

```
1  raise ValueError("Invalid data") from e
```

Interview answer:

Exception chaining is used to show that one exception was caused by another. It helps with debugging by preserving the original error context.

7. I/O Operations

46. How do you read a file in Python?

```
1 with open("file.txt", "r") as f:  
2     data = f.read()
```

Interview answer:

We can read a file using the `open()` function. The recommended way is using the `with` statement because it automatically closes the file.

47. Difference between `read()` and `readlines()`.

`read()` returns the entire file as a string.

`readlines()` returns a list of lines.

Interview answer:

`read()` reads the whole file content at once, while `readlines()` reads the file and returns each line as an element in a list.

48. How do you write data into a file?

```
1 with open("file.txt", "w") as f:  
2     f.write("Hello")
```

Interview answer:

We can write to a file using write mode `"w"` or append mode `"a"`. The `with` statement ensures the file is closed properly.

49. What is the use of `with` statement?

It manages resources automatically.

Interview answer:

The `with` statement is used for context management. It automatically handles setup and cleanup, such as closing files or database connections.

50. How does Python handle file buffering?

Python buffers file operations to improve performance.

Interview answer:

Python does not always write data immediately to disk. It stores data temporarily in a buffer and writes it in chunks for better performance.

51. How would you safely read a large file?

```
1 with open("large.txt") as f:
2     for line in f:
3         process(line)
```

Interview answer:

For large files, I would read line by line instead of loading the whole file into memory.

8. Object-Oriented Python

52. What is a class and an object?

A class is a blueprint. An object is an instance.

```
1 class Person:
2     pass
3
4 p = Person()
```

Interview answer:

A class defines structure and behavior, while an object is a real instance of that class.

53. Explain `__init__`.

`__init__` is a constructor method.

```
1 class Person:
2     def __init__(self, name):
3         self.name = name
```

Interview answer:

`__init__` is automatically called when an object is created. It is used to initialize object attributes.

54. What is inheritance?

Inheritance allows one class to reuse another class's properties and methods.

```
1 class Dog(Animal):
2     pass
```

Interview answer:

Inheritance allows a child class to acquire behavior from a parent class. It promotes code reuse.

55. Difference between class variable and instance variable.

Class variable is shared by all objects.

Instance variable belongs to a specific object.

```
1 class Student:
2     school = "ABC"          # class variable
3
4     def __init__(self, name):
5         self.name = name    # instance variable
```

Interview answer:

Class variables are shared across all instances, while instance variables are unique to each object.

56. What is multiple inheritance?

A class inherits from more than one parent class.

```
1 class C(A, B):
2     pass
```

Interview answer:

Multiple inheritance allows a class to inherit features from multiple parent classes.

57. Explain Method Resolution Order, MRO.

MRO decides the order in which Python searches methods.

Interview answer:

MRO defines the order in which parent classes are searched when calling a method. Python uses the C3 linearization algorithm for MRO.

9. Database Using Python

58. What is CRUD?

CRUD means:

- Create
- Read
- Update
- Delete

Interview answer:

CRUD represents the four basic database operations: creating, reading, updating, and deleting records.

59. How do you connect Python to a database?

Using database drivers.

Examples:

```
1  sqlite3
2  psycopg2
3  mysql.connector
4  SQLAlchemy
```

Interview answer:

Python connects to databases using database drivers or ORM libraries. For example, sqlite3 for SQLite, psycopg2 for PostgreSQL, and SQLAlchemy as an ORM.

60. What is an ORM?

ORM means Object Relational Mapper.

Interview answer:

An ORM allows developers to interact with databases using Python classes and objects instead of writing raw SQL queries.

61. Difference between SQLite and PostgreSQL.

SQLite is lightweight and file-based. PostgreSQL is a powerful client-server database.

Interview answer:

SQLite is suitable for small applications, local storage, and testing. PostgreSQL is better for production applications requiring scalability, concurrency, and advanced database features.

62. How do you handle database transactions?

```
1  try:
2      conn.commit()
3  except:
4      conn.rollback()
```

Interview answer:

Transactions are handled using commit and rollback. If all operations succeed, we commit. If an error occurs, we rollback to maintain data consistency.

63. Explain connection pooling.

Connection pooling reuses database connections.

Interview answer:

Connection pooling keeps a pool of active database connections and reuses them instead of creating a new connection every time. This improves performance.

10. Generators, Comprehensions, Lambda

64. What is a lambda function?

```
1  square = lambda x: x * x
```

Interview answer:

A lambda function is an anonymous one-line function used for simple operations.

65. What is list comprehension?

```
1  squares = [x*x for x in range(5)]
```

Interview answer:

List comprehension is a concise way to create lists using a loop and optional condition.

66. Difference between generator and list.

A list stores all values in memory.

A generator produces values one at a time.

Interview answer:

Generators are memory efficient because they generate values lazily, whereas lists store all elements in memory.

67. What is yield?

yield returns a value and pauses the function.

```
1 def gen():
2     yield 1
3     yield 2
```

Interview answer:

yield is used in generator functions. It returns a value but saves the function state so execution can continue from the same point later.

68. How do generators improve performance?

They save memory and compute values only when needed.

Interview answer:

Generators improve performance by avoiding loading all data into memory. They are useful for large files, streams, and large datasets.

69. Write a custom generator example.

```
1 def count_up_to(n):
2     count = 1
3     while count <= n:
4         yield count
5         count += 1
```

Interview answer:

This generator produces numbers one at a time instead of creating a full list in memory.

11. Regex, Logging, Unit Testing

70. What is a regular expression?

A regular expression is a pattern used to search or validate text.

```
1 import re
2
3 re.match(r"\d+", "123abc")
```

Interview answer:

Regex is used for pattern matching, validation, searching, and replacing text.

71. Why is logging important?

Logging helps track application behavior.

Interview answer:

Logging is important for debugging, monitoring, auditing, and troubleshooting production issues.

72. Difference between print and logging.

print is simple console output.

Logging is configurable and production-friendly.

Interview answer:

print is mainly for quick debugging, while logging supports levels, timestamps, files, formatting, and production monitoring.

73. What are test cases?

Test cases verify code behavior.

Interview answer:

A test case checks whether a specific part of the application works as expected under given conditions.

74. Explain log levels.

Common levels:

```
1 DEBUG
```

- | | |
|---|----------|
| 2 | INFO |
| 3 | WARNING |
| 4 | ERROR |
| 5 | CRITICAL |

Interview answer:

Log levels indicate the severity of messages. DEBUG is detailed information, while CRITICAL indicates serious failures.

75. How do you mock objects in unit testing?

1	<code>from unittest.mock import Mock</code>
---	---

Interview answer:

Mocking is used to replace real objects or external dependencies during testing, such as APIs, databases, or file systems.

12. NumPy, Pendulum, Requests, Pandas, Pytest

76. Difference between NumPy array and list.

NumPy arrays are faster and support vectorized operations.

Interview answer:

Python lists can store mixed data types, while NumPy arrays usually store homogeneous data and are optimized for numerical operations.

77. What is a DataFrame?

A DataFrame is a tabular data structure from Pandas.

Interview answer:

A DataFrame is like an Excel table or SQL table with rows and columns. It is used for data analysis and manipulation.

78. How do you make a GET/POST request?

1	<code>import requests</code>
2	
3	<code>response = requests.get("https://api.example.com")</code>
1	<code>response = requests.post("https://api.example.com",</code>
	<code>json={"name": "Vijay"})</code>

Interview answer:

Using the requests library, we can make HTTP calls such as GET, POST, PUT, and DELETE.

79. Difference between unittest and pytest.

unittest is built-in. pytest is more flexible and concise.

Interview answer:

unittest follows an object-oriented style and is part of the standard library. pytest provides simpler syntax, fixtures, plugins, and better test discovery.

80. What is vectorization?

Vectorization means applying operations to entire arrays without explicit loops.

```
1 arr * 2
```

Interview answer:

Vectorization improves performance by using optimized low-level operations instead of Python loops.

81. How does fixture work in pytest?

Fixtures provide reusable setup and teardown logic.

```
1 import pytest
2
3 @pytest.fixture
4 def sample_data():
5     return [1, 2, 3]
```

Interview answer:

Fixtures in pytest are used to prepare test data or resources before tests run. They help avoid duplicate setup code.

13. Advanced Python Concepts

82. How would you optimize slow Python code?

Steps:

1. Profile first
2. Use better algorithms

3. Use built-in functions
4. Use comprehensions/generators
5. Avoid unnecessary loops
6. Use caching
7. Use NumPy/Pandas for data-heavy work
8. Use multiprocessing/async where suitable

Interview answer:

I would first profile the code to find bottlenecks, then optimize algorithms, reduce unnecessary computations, use efficient data structures, caching, built-in functions, and libraries like NumPy where applicable.

83. How do you profile a Python application?

```
1 python -m cProfile app.py
```

Interview answer:

Python applications can be profiled using tools like cProfile, timeit, line_profiler, and memory profilers. Profiling helps identify slow functions and performance bottlenecks.

84. Explain garbage collection in Python.

Python uses reference counting and cyclic garbage collection.

Interview answer:

Python automatically manages memory. It uses reference counting to remove unused objects and a garbage collector to clean circular references.

14. Flask Basics

85. What is Flask?

Flask is a lightweight Python web framework.

Interview answer:

Flask is a micro web framework used to build web applications and APIs. It is simple, flexible, and gives developers control over components.

86. What is routing?

Routing maps URLs to functions.

```
1 @app.route("/home")
2 def home():
3     return "Home"
```

Interview answer:

Routing defines which function should run when a user accesses a specific URL.

87. What are Blueprints?

Blueprints organize Flask applications into modules.

Interview answer:

Blueprints help split a large Flask application into smaller reusable components, such as user routes, admin routes, or API routes.

88. How do templates work?

Flask uses Jinja2 templates.

Interview answer:

Templates allow dynamic HTML generation. Flask passes data from Python to templates, and Jinja2 renders the final HTML.

89. Flask vs Django.

Flask is lightweight and flexible. Django is full-featured.

Interview answer:

Flask is suitable when we need flexibility and minimal setup. Django includes built-in features like ORM, admin panel, authentication, and is better for large structured applications.

90. How does Flask handle requests internally?

Flask uses Werkzeug WSGI toolkit.

Interview answer:

When a request comes in, Flask matches the URL with a route, creates request and application contexts, executes the view function, and returns a response.

15. REST API

91. What is REST?

REST stands for Representational State Transfer.

Interview answer:

REST is an architectural style for designing APIs. It uses HTTP methods, resources, URLs, and stateless communication.

92. What are HTTP methods?

Common HTTP methods:

- | | | |
|---|--------|-------------------------|
| 1 | GET | - read data |
| 2 | POST | - create data |
| 3 | PUT | - replace data |
| 4 | PATCH | - partially update data |
| 5 | DELETE | - delete data |

93. Difference between PUT and PATCH.

PUT replaces the whole resource.

PATCH updates part of the resource.

Interview answer:

PUT is used for full updates, while PATCH is used for partial updates.

94. What are status codes?

Status codes indicate HTTP response result.

Examples:

- | | | |
|---|-----|-----------------------|
| 1 | 200 | OK |
| 2 | 201 | Created |
| 3 | 400 | Bad Request |
| 4 | 401 | Unauthorized |
| 5 | 403 | Forbidden |
| 6 | 404 | Not Found |
| 7 | 500 | Internal Server Error |

95. How do you handle versioning?

Common ways:

```
1 /api/v1/users
2 /api/v2/users
```

or headers.

Interview answer:

API versioning can be handled through URL paths, headers, or query parameters. URL versioning is commonly used because it is simple and clear.

96. What is idempotency?

Repeated requests produce the same result.

Interview answer:

An idempotent operation gives the same result even if executed multiple times. GET, PUT, and DELETE are generally idempotent, while POST is usually not.

16. Database in Web Context

97. What is SQLAlchemy?

SQLAlchemy is a Python SQL toolkit and ORM.

Interview answer:

SQLAlchemy allows Python applications to interact with relational databases using ORM models or SQL expressions.

98. What are migrations?

Migrations track database schema changes.

Interview answer:

Migrations are used to version and apply database schema changes safely, such as creating tables or adding columns. In Flask, Alembic or Flask-Migrate is commonly used.

99. How do you prevent SQL injection?

Use parameterized queries or ORM.

```
1 cursor.execute("SELECT * FROM users WHERE id = ?", (user_id,))
```

Interview answer:

SQL injection can be prevented by using parameterized queries, prepared statements, ORM methods, input validation, and avoiding string concatenation in SQL queries.

17. Decorators

100. What is a decorator?

A decorator modifies the behavior of a function.

```
1 @decorator
2 def func():
3     pass
```

Interview answer:

A decorator is a function that takes another function as input and extends or modifies its behavior without changing its source code.

101. Why are decorators used?

They are used for reusable logic.

Examples:

- Logging
 - Authentication
 - Authorization
 - Caching
 - Timing
 - Validation
-

102. How do decorators work internally?

```
1 @my_decorator
2 def hello():
3     pass
```

is same as:

```
1 hello = my_decorator(hello)
```

Interview answer:

Internally, a decorator wraps the original function with another function and returns the wrapper.

103. Write a decorator for logging.

```
1 def log_decorator(func):
2     def wrapper(*args, **kwargs):
3         print(f"Calling function: {func.__name__}")
4         result = func(*args, **kwargs)
5         print(f"Finished function: {func.__name__}")
6         return result
7     return wrapper
8
9 @log_decorator
10 def add(a, b):
11     return a + b
```

18. Docker

104. What is Docker?

Docker is a containerization platform.

Interview answer:

Docker packages an application with its dependencies into a container so it can run consistently across different environments.

105. Difference between image and container.

Image is a blueprint. Container is a running instance.

Interview answer:

A Docker image is a read-only template containing application code and dependencies. A container is a running instance of that image.

106. What is a Dockerfile?

A Dockerfile contains instructions to build an image.

```
1 FROM python:3.11
2 COPY . /app
3 WORKDIR /app
4 RUN pip install -r requirements.txt
```

```
5 CMD ["python", "app.py"]
```

107. What are volumes?

Volumes persist data outside containers.

Interview answer:

Docker volumes are used to store data persistently even if the container is removed or recreated.

108. How does Docker networking work?

Docker allows containers to communicate through networks.

Interview answer:

Docker networking enables communication between containers, the host, and external systems. Common network types are bridge, host, and overlay.

109. Docker vs VM.

Docker containers share the host OS kernel. VMs include a full OS.

Interview answer:

Docker is lightweight and faster because containers share the host OS kernel. Virtual machines are heavier because each VM runs its own full operating system.

19. FastAPI

110. Why FastAPI?

FastAPI is fast, modern, and supports async.

Interview answer:

FastAPI is used for building high-performance APIs. It supports async programming, automatic validation, type hints, and auto-generated Swagger documentation.

111. What is Pydantic?

Pydantic validates data using Python type hints.

```
1 from pydantic import BaseModel
2
```

```
3 class User(BaseModel):
4     name: str
5     age: int
```

Interview answer:

Pydantic is used in FastAPI for request validation, serialization, and data parsing based on Python type annotations.

112. How does async work in FastAPI?

```
1 @app.get("/")
2 async def home():
3     return {"message": "Hello"}
```

Interview answer:

FastAPI supports asynchronous endpoints using `async def`. This improves performance for I/O-bound operations such as database calls, API calls, and file operations.

113. What is dependency injection?

Dependency injection provides required objects/functions to routes.

```
1 def get_db():
2     return db
3
4 @app.get("/users")
5 def users(db=Depends(get_db)):
6     pass
```

Interview answer:

Dependency injection in FastAPI is used to share reusable logic like database connections, authentication, and configuration across routes.

114. FastAPI vs Flask performance.

FastAPI is generally faster due to ASGI and async support.

Interview answer:

FastAPI usually performs better for async and high-concurrency APIs because it is built on ASGI. Flask is WSGI-based and synchronous by default, though still good for many applications.

115. How does FastAPI auto-generate Swagger docs?

FastAPI uses type hints and OpenAPI schema.

Interview answer:

FastAPI reads path operations, type hints, request models, and response models to automatically generate OpenAPI documentation, which is shown through Swagger UI.

Additional Interview Questions & Answers

Python Core Additional Questions

116. What is None in Python?

None represents absence of a value.

Answer:

None is a special singleton object in Python used to represent no value or null value.

117. What is the difference between shallow copy and deep copy?

```
1  import copy
2
3  copy.copy(obj)      # shallow copy
4  copy.deepcopy(obj)  # deep copy
```

Answer:

A shallow copy copies only the outer object, while nested objects are still shared. A deep copy recursively copies all nested objects.

118. What is Python indentation?

Python uses indentation to define blocks of code.

Answer:

Unlike languages that use braces, Python uses indentation to group statements inside loops, functions, classes, and conditions.

119. What is PEP 8?

PEP 8 is Python's style guide.

Answer:

PEP 8 provides coding conventions for writing readable and consistent Python code.

120. What is the Global Interpreter Lock, GIL?

GIL allows only one thread to execute Python bytecode at a time.

Answer:

The GIL is a lock in CPython that prevents multiple native threads from executing Python bytecode simultaneously. It affects CPU-bound multithreaded programs.

OOP Additional Questions

121. What is encapsulation?

Encapsulation means wrapping data and methods together.

Answer:

Encapsulation protects object data by controlling access through methods and attributes.

122. What is polymorphism?

Polymorphism means same interface, different behavior.

```
1 len("abc")
2 len([1, 2, 3])
```

Answer:

Polymorphism allows different objects to respond to the same method or operation in different ways.

123. What is abstraction?

Abstraction hides implementation details.

Answer:

Abstraction exposes only necessary functionality while hiding internal implementation complexity.

124. What are magic methods?

Magic methods are special methods with double underscores.

Examples:

```
1  __init__  
2  __str__  
3  __len__  
4  __add__
```

Answer:

Magic methods allow custom behavior for built-in operations like object creation, printing, length, addition, and comparison.

API Additional Questions

125. What is authentication vs authorization?

Authentication verifies identity. Authorization checks permission.

Answer:

Authentication confirms who the user is, while authorization determines what the user is allowed to access.

126. What is JWT?

JWT means JSON Web Token.

Answer:

JWT is a token-based authentication mechanism used to securely transmit user identity and claims between client and server.

127. What is middleware?

Middleware runs before or after request processing.

Answer:

Middleware is used for common processing like logging, authentication, CORS, request validation, and error handling.

128. What is CORS?

CORS means Cross-Origin Resource Sharing.

Answer:

CORS is a browser security mechanism that controls whether resources from one domain can be accessed by another domain.

129. What is pagination?

Pagination divides large data into smaller pages.

Answer:

Pagination improves performance and user experience by returning limited records per request.

Database Additional Questions

130. What is an index in a database?

An index improves query speed.

Answer:

A database index is a data structure that speeds up data retrieval but may slow down insert/update operations because the index also needs updating.

131. What is normalization?

Normalization organizes data to reduce redundancy.

Answer:

Normalization splits data into related tables to avoid duplication and improve data integrity.

132. What is a primary key?

A primary key uniquely identifies a record.

Answer:

A primary key is a unique, non-null column used to identify each row in a table.

133. What is a foreign key?

A foreign key links two tables.

Answer:

A foreign key is a column that references the primary key of another table to maintain relationships.

134. What is ACID?

ACID stands for:

- Atomicity
- Consistency
- Isolation
- Durability

Answer:

ACID properties ensure reliable database transactions.

Docker Additional Questions

135. What is Docker Compose?

Docker Compose manages multiple containers.

Answer:

Docker Compose uses a YAML file to define and run multi-container applications such as web app, database, and cache together.

136. What is .dockerignore?

.dockerignore excludes files from Docker build context.

Answer:

It works like .gitignore and helps reduce image size and build time.

137. How do you reduce Docker image size?

Use:

- Smaller base images
- .dockerignore
- Multi-stage builds
- Remove unnecessary packages

- Combine RUN commands carefully

Answer:

Docker image size can be reduced by using slim images, multi-stage builds, and excluding unnecessary files.

Good Interview Closing Answer

If the interviewer asks, **“How strong are you in Python backend?”**, you can say:

I have a good understanding of Python fundamentals, data structures, functions, exception handling, OOP, file handling, database operations, REST APIs, Flask/FastAPI, testing, logging, and Docker. I can build APIs, connect them with databases, write clean reusable code, handle errors properly, and write unit tests. I also understand performance optimization using profiling, generators, vectorization, and efficient data structures.

If you want, I can also prepare this as a **Python interview cheat sheet with only short answers**, or a **mock interview Q&A format for 2–4 years Python backend developer role**.

SQL topics:

1. DCL (Data Control Language)

Used for **permission / access control**.

Commands:

- *GRANT* → give privileges
- *REVOKE* → remove privileges

☒ Example:

```
1 GRANT SELECT, INSERT ON employees TO user1;  
2 REVOKE INSERT ON employees FROM user1;
```

2. TCL (Transaction Control Language)

Used to **manage transactions** in database.

Commands:

- *COMMIT* → save changes permanently
- *ROLLBACK* → undo changes
- *SAVEPOINT* → set point to rollback partially

☒ Example:

```
1 BEGIN;  
2 UPDATE acc SET balance = balance - 500 WHERE id=1;  
3 SAVEPOINT sp1;  
4 UPDATE acc SET balance = balance + 500 WHERE id=2;  
5 ROLLBACK TO sp1;  
6 COMMIT;
```

3. JOINS

Used to **combine data from multiple tables**.

Types:

- *INNER JOIN* → common records
- *LEFT JOIN* → all left + matching right
- *RIGHT JOIN* → all right + matching left
- *FULL JOIN* → all records
- *CROSS JOIN* → Cartesian product

☒ Example:

```
1 SELECT e.name, d.name
2 FROM emp e
3 INNER JOIN dept d ON e.dept_id = d.id;
```

4. ACID Properties

Ensures **reliable transactions**.

- **A – Atomicity** → all or nothing
 - **C – Consistency** → valid state
 - **I – Isolation** → no interference
 - **D – Durability** → permanent data
-

5. Normalization

Process of **removing redundancy**.

Types:

- **1NF** → no repeating groups
 - **2NF** → no partial dependency
 - **3NF** → no transitive dependency
-

6. WHERE Clause

Used to **filter rows**.

☒ Example:

```
1 SELECT * FROM emp WHERE salary > 50000;
```

7. WINDOW Functions

Perform calculations **over a set of rows**.

☒ Example:

```
1 SELECT name, salary,
```

```
2 RANK() OVER (ORDER BY salary DESC) AS rank
3 FROM emp;
```

8. SET Operations

Combine results of two queries.

Types:

- *UNION* → removes duplicates
- *UNION ALL* → keeps duplicates
- *INTERSECT* → common rows
- *EXCEPT / MINUS* → difference

☒ Example:

```
1 SELECT name FROM emp1
2 UNION
3 SELECT name FROM emp2;
```

9. Aggregate Functions

Used to **perform calculations**.

- *COUNT()*
- *SUM()*
- *AVG()*
- *MIN()*
- *MAX()*

☒ Example:

```
1 SELECT AVG(salary) FROM emp;
```

10. Transactions

Group of SQL operations executed as **one unit**.

☒ Example:

```
1 BEGIN;
```

```
2 UPDATE accounts SET balance = balance - 100;
3 UPDATE accounts SET balance = balance + 100;
4 COMMIT;
```

11. Queries (Types)

- *DDL* → CREATE, ALTER, DROP
 - *DML* → INSERT, UPDATE, DELETE
 - *DQL* → SELECT
 - *DCL, TCL*
-

12. Second Highest / Lowest Salary

☒ Second Highest

```
1 SELECT MAX(salary)
2 FROM emp
3 WHERE salary < (SELECT MAX(salary) FROM emp);
```

☒ Second Lowest

```
1 SELECT MIN(salary)
2 FROM emp
3 WHERE salary > (SELECT MIN(salary) FROM emp);
```

13. REPLACE

Used to **replace string data**.

☒ Example:

```
1 SELECT REPLACE('Hello SQL', 'SQL', 'World');
2 -- Output: Hello World
```

☒ Quick Summary Tip:

- DCL → permissions
- TCL → transactions
- JOIN → combine tables

- ACID → reliability
 - Normalization → remove duplication
 - WHERE → filter
 - WINDOW → advanced analytics
 - SET → combine queries
 - AGG → calculations
-

Basic but important Python coding interview questions with clean solutions. These are commonly asked for fresher/entry-level Python interviews.

1. Reverse a String

```
1 s = "python"
2 print(s[::-1])
```

Output:

```
1 nohtyp
```

2. Check if a String is Palindrome

```
1 def is_palindrome(s):
2     return s == s[::-1]
3
4 print(is_palindrome("madam"))
5 print(is_palindrome("hello"))
```

Output:

```
1 True
2 False
```

3. Find Factorial of a Number

```
1 def factorial(n):
2     result = 1
3     for i in range(1, n + 1):
4         result *= i
5     return result
6
7 print(factorial(5))
```

Output:

```
1 120
```

4. Fibonacci Series

```
1 def fibonacci(n):
2     a, b = 0, 1
3
4     for _ in range(n):
5         print(a, end=" ")
6         a, b = b, a + b
7
8 fibonacci(7)
```

Output:

```
1 0 1 1 2 3 5 8
```

5. Check Prime Number

```
1 def is_prime(n):
2     if n <= 1:
3         return False
4
5     for i in range(2, int(n ** 0.5) + 1):
6         if n % i == 0:
7             return False
8
9     return True
10
11 print(is_prime(7))
12 print(is_prime(10))
```

Output:

```
1 True
2 False
```

6. Find Largest Number in a List

```
1 numbers = [10, 25, 7, 90, 45]
2
3 largest = numbers[0]
4
5 for num in numbers:
```

```
6     if num > largest:
7         largest = num
8
9     print(largest)
```

Output:

```
1  90
```

7. Find Second Largest Number in a List

```
1  numbers = [10, 25, 7, 90, 45]
2
3  unique_numbers = list(set(numbers))
4  unique_numbers.sort()
5
6  print(unique_numbers[-2])
```

Output:

```
1  45
```

8. Count Vowels in a String

```
1  def count_vowels(s):
2      vowels = "aeiouAEIOU"
3      count = 0
4
5      for char in s:
6          if char in vowels:
7              count += 1
8
9      return count
10
11 print(count_vowels("Python Interview"))
```

Output:

```
1  5
```

9. Count Frequency of Characters

```
1 s = "programming"
2
3 frequency = {}
4
5 for char in s:
6     frequency[char] = frequency.get(char, 0) + 1
7
8 print(frequency)
```

Output:

```
1 {'p': 1, 'r': 2, 'o': 1, 'g': 2, 'a': 1, 'm': 2, 'i': 1, 'n': 1}
```

10. Remove Duplicates from a List

```
1 numbers = [1, 2, 2, 3, 4, 4, 5]
2
3 unique = []
4
5 for num in numbers:
6     if num not in unique:
7         unique.append(num)
8
9 print(unique)
```

Output:

```
1 [1, 2, 3, 4, 5]
```

11. Check Anagram Strings

Two strings are anagrams if they contain the same characters.

```
1 def are_anagrams(s1, s2):
2     return sorted(s1) == sorted(s2)
3
4 print(are_anagrams("listen", "silent"))
5 print(are_anagrams("hello", "world"))
```

Output:

```
1 True
```

12. Find Missing Number in a List

Example: numbers from 1 to 5, but one number is missing.

```
1 numbers = [1, 2, 4, 5]
2
3 n = 5
4 expected_sum = n * (n + 1) // 2
5 actual_sum = sum(numbers)
6
7 missing = expected_sum - actual_sum
8
9 print(missing)
```

Output:

```
1 3
```

13. Swap Two Numbers Without Third Variable

```
1 a = 10
2 b = 20
3
4 a, b = b, a
5
6 print(a, b)
```

Output:

```
1 20 10
```

14. Check Even or Odd

```
1 num = 8
2
3 if num % 2 == 0:
4     print("Even")
5 else:
6     print("Odd")
```

Output:

```
1 Even
```

15. Find Common Elements in Two Lists

```
1 list1 = [1, 2, 3, 4]
2 list2 = [3, 4, 5, 6]
3
4 common = []
5
6 for item in list1:
7     if item in list2:
8         common.append(item)
9
10 print(common)
```

Output:

```
1 [3, 4]
```

16. Reverse a List

```
1 numbers = [1, 2, 3, 4, 5]
2
3 print(numbers[::-1])
```

Output:

```
1 [5, 4, 3, 2, 1]
```

17. Find Sum of Digits

```
1 num = 12345
2 total = 0
3
4 while num > 0:
5     digit = num % 10
6     total += digit
7     num = num // 10
8
```

```
9 print(total)
```

Output:

```
1 15
```

18. Check Armstrong Number

Example:

$$153 = 1^3 + 5^3 + 3^3 = 153$$

```
1 def is_armstrong(num):
2     digits = str(num)
3     power = len(digits)
4
5     total = 0
6
7     for digit in digits:
8         total += int(digit) ** power
9
10    return total == num
11
12 print(is_armstrong(153))
13 print(is_armstrong(123))
```

Output:

```
1 True
2 False
```

19. Find Duplicate Elements in a List

```
1 numbers = [1, 2, 3, 2, 4, 5, 1]
2
3 duplicates = []
4
5 for num in numbers:
6     if numbers.count(num) > 1 and num not in duplicates:
7         duplicates.append(num)
8
9 print(duplicates)
```

Output:

```
1 [1, 2]
```

20. Check if Two Lists are Equal

```
1 list1 = [1, 2, 3]
2 list2 = [3, 2, 1]
3
4 if sorted(list1) == sorted(list2):
5     print("Equal")
6 else:
7     print("Not Equal")
```

Output:

```
1 Equal
```

21. Find Maximum and Minimum in a List

```
1 numbers = [10, 20, 5, 40, 2]
2
3 maximum = max(numbers)
4 minimum = min(numbers)
5
6 print("Maximum:", maximum)
7 print("Minimum:", minimum)
```

Output:

```
1 Maximum: 40
2 Minimum: 2
```

22. Count Words in a Sentence

```
1 sentence = "Python is easy to learn"
2
3 words = sentence.split()
4
5 print(len(words))
```

Output:

```
1 5
```

23. Remove Spaces from a String

```
1 s = "Python is easy"
2
3 result = s.replace(" ", "")
4
5 print(result)
```

Output:

```
1 Pythoniseasy
```

24. Find Length of String Without *len()*

```
1 s = "python"
2
3 count = 0
4
5 for char in s:
6     count += 1
7
8 print(count)
```

Output:

```
1 6
```

25. Print Numbers from 1 to N

```
1 n = 5
2
3 for i in range(1, n + 1):
4     print(i)
```

Output:

```
1 1
2 2
3 3
4 4
5 5
```

26. Check Leap Year

```
1 year = 2024
2
3 if (year % 400 == 0) or (year % 4 == 0 and year % 100 != 0):
4     print("Leap Year")
5 else:
6     print("Not Leap Year")
```

Output:

```
1 Leap Year
```

27. Find Character with Maximum Frequency

```
1 s = "programming"
2
3 frequency = {}
4
5 for char in s:
6     frequency[char] = frequency.get(char, 0) + 1
7
8 max_char = max(frequency, key=frequency.get)
9
10 print(max_char)
```

Output:

```
1 r
```

Note: *r*, *g*, and *m* all appear twice. Python returns the first maximum key found based on dictionary order.

28. Sort a List Without Using *sort()*

```
1 numbers = [5, 2, 8, 1, 3]
2
3 for i in range(len(numbers)):
4     for j in range(i + 1, len(numbers)):
```

```
5         if numbers[i] > numbers[j]: numbers[i], numbers[j] =  
numbers[j], numbers[i]  
6  
7 print(numbers)
```

Output:

```
1 [1, 2, 3, 5, 8]
```

29. Merge Two Dictionaries

```
1 dict1 = {"a": 1, "b": 2}  
2 dict2 = {"c": 3, "d": 4}  
3  
4 merged = {**dict1, **dict2}  
5  
6 print(merged)
```

Output:

```
1 {'a': 1, 'b': 2, 'c': 3, 'd': 4}
```

30. Find First Non-Repeating Character

```
1 s = "aabbcd"e"  
2  
3 for char in s:  
4     if s.count(char) == 1:  
5         print(char)  
6         break
```

Output:

```
1 c
```

Most Important Topics to Practice

For Python coding interviews, focus on:

1. Strings
2. Lists
3. Dictionaries
4. Loops
5. Functions

6. Recursion
 7. Searching and sorting
 8. Basic mathematics
 9. Pattern printing
 10. Object-oriented programming basics
-

Quick Interview Tip

When solving coding questions, always explain:

1. What the problem is asking
2. Your logic
3. Your code
4. Time complexity
5. Edge cases

Example:

```
1 def reverse_string(s):  
2     return s[::-1]
```

Time complexity: $O(n)$

Space complexity: $O(n)$